

TP 4 : Middleware spécialisé

I. Introduction

Ce TP porte sur l'étude des middlewares spécialisés pour l'Internet des Objets. Il permet de manipuler la plateforme Home Assistant. L'objectif est de comprendre l'architecture d'un système domotique et d'apprendre à intégrer des dispositifs en utilisant le protocole de communication MQTT. À travers la configuration d'un microcontrôleur ESP8266, nous avons pu piloter une LED et lire un capteur de luminosité pour comprendre les concepts d'entités, d'états et de services.

II. Découverte de Home Assistant

A. Interface et concepts de base

La première partie de ce TP a consisté à se connecter à l'interface de Home Assistant afin d'explorer le tableau de bord principal et les outils de développement. Les entités sont les objets qui représentent les composants du système (capteurs, actionneurs...). Les domaines servent à les catégoriser (light, sensor, person). Nous avons également saisi la différence entre l'état d'une entité (sa valeur actuelle) et ses attributs (les métadonnées associées). Il est possible d'utiliser des services pour déclencher des actions sur les entités.

Nous avons pu lister les entités actives sur le serveur, comme le montre la figure suivante :

Entité	État	Attributs
 conversation.home_assistant ① Home Assistant	unknown	friendly_name: Home Assistant supported_features: 1
 event.backup_automatic_backup ① Backup Sauvegarde automatique	unknown	event_types: completed, failed, in_progress event_type: null friendly_name: Backup Sauvegarde automatique
 person.insa ① insa	unknown	editable: true id: insa device_trackers: user_id: 6b2b377dbb0467ea0703a8bf0c6af54 friendly_name: insa
 person.user ① user	unknown	editable: true id: user device_trackers: user_id: 86843e254d5c4ddb6251a47101db80a friendly_name: user
 sensor.backup_backup_manager_state ① Backup État du gestionnaire de sauvegarde	idle	options: idle, create_backup, blocked, receive_backup, restore_backup device_class: enum friendly_name: Backup État du gestionnaire de sauvegarde
 sensor.backup_last_attempted_automatic_backup ① Backup Dernière tentative de sauvegarde automatique	unknown	device_class: timestamp friendly_name: Backup Dernière tentative de sauvegarde automatique
 sensor.backup_last_successful_automatic_backup ① Backup Dernière sauvegarde automatique réussie	unknown	device_class: timestamp friendly_name: Backup Dernière sauvegarde automatique réussie
 sensor.backup_next_scheduled_automatic_backup ① Backup Prochaine sauvegarde automatique programmée	unknown	device_class: timestamp friendly_name: Backup Prochaine sauvegarde automatique programmée
 sensor.sun_next_dawn ① Sun Prochaine aube	2026-01-07T07:07:47+00:00	device_class: timestamp friendly_name: Sun Prochaine aube
 sensor.sun_next_dusk ① Sun Prochain crépuscule	2026-01-06T16:24:43+00:00	device_class: timestamp friendly_name: Sun Prochain crépuscule
 sensor.sun_next_midnight ① Sun Prochain minuit	2026-01-06T23:46:43+00:00	device_class: timestamp friendly_name: Sun Prochain minuit
 sensor.sun_next_noon ① Sun Prochain midi	2026-01-06T11:46:03+00:00	device_class: timestamp friendly_name: Sun Prochain midi
 sensor.sun_next_rising ① Sun Prochain lever	2026-01-07T07:49:03+00:00	device_class: timestamp friendly_name: Sun Prochain lever
 sensor.sun_next_setting ① Sun Prochain coucher	2026-01-06T15:43:19+00:00	device_class: timestamp friendly_name: Sun Prochain coucher

Figure 1 : Entités et leurs attributs.

L'entité `person.insa` dont l'état est actuellement `unknown` possède des attributs détaillés tels que son identifiant (`id: insa`) et son nom "friendly" (`friendly_name: insa`).

Des capteurs comme `sensor.sun_next_dawn` fournit une dernière valeur temporelle de l'état et définit sa catégorie avec l'attribut `device_class: timestamp`.

Pour tester l'interaction avec le système, nous avons utilisé le service `persistent_notification.create`. Nous avons personnalisé l'action avec le titre "Chaban : Mon premier test" et le message "Hello de Chaban depuis les outils de développement !".

Outils de développement

YAML États Actions Modèle Événements Statistiques Assist

L'outil de développement d'actions vous permet d'effectuer n'importe quelle action disponible dans Home Assistant.

Créer `persistent_notification.create`

Affiche une notification sur le panneau de notifications.

Message
Corps du message de la notification.

Hello de Chaban depuis les outils de développement !

Titre
Titre facultatif de la notification.

Chaban : Mon premier test

ID de notification
ID de la notification. Cette nouvelle notification écrasera une notification existante avec le même ID.

Passez en mode YAML

Exécuter l'action

Figure 2 : Création de l'action "Chaban : Mon premier test".

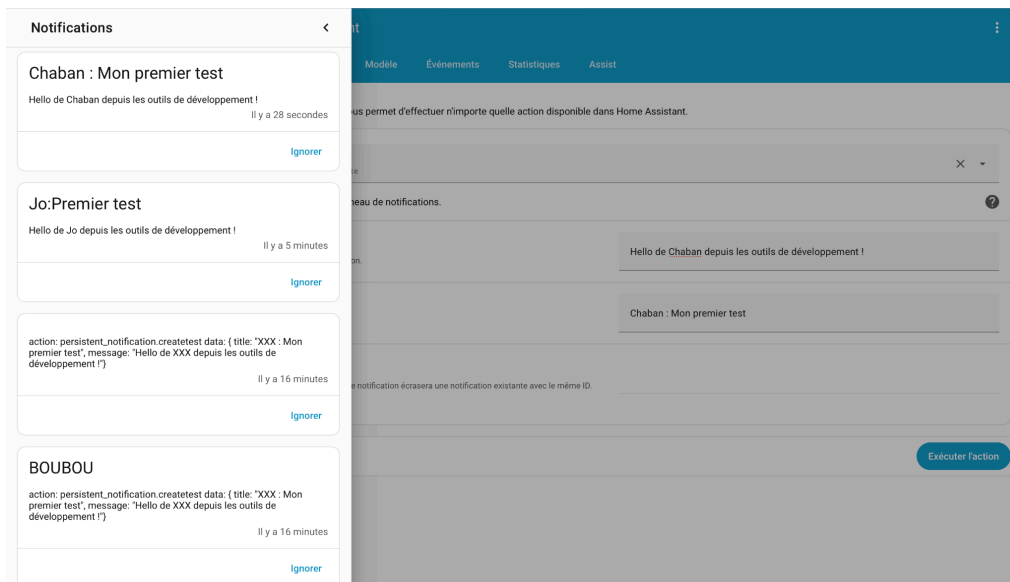


Figure 3 : Exécution de l'action "Chaban : Mon premier test".

Après avoir cliqué sur *Exécuter l'action*, nous avons bien la notification validant l'exécution de l'action. Nous avons également accès aux autres messages envoyés.

B. Configuration MQTT

Cette étape consiste à vérifier l'intégration du broker MQTT (Mosquitto) au sein de Home Assistant pour permettre la communication avec des dispositifs IoT. Après avoir validé la présence de l'intégration dans le menu "Appareils et services", nous avons effectué un test de communication bidirectionnelle (boucle locale) directement depuis l'interface de configuration MQTT. Nous avons configuré l'outil pour écouter le topic *test/chaban*. Un paquet a été publié avec la payload "Hello de Chaban". Dans la section d'écoute, on constate la réception immédiate du message : "Message 0 reçu sur test/chaban à 10:56 : Hello de Chaban". Le middleware est désormais prêt à recevoir les données provenant de l'ESP8266.

The screenshot displays the Home Assistant MQTT configuration interface. It is divided into two main sections: 'Publier un paquet' (Publish a packet) and 'Écouter un sujet' (Listen to a topic).

In the 'Publier un paquet' section, the 'Sujet' (Topic) is set to 'test/chaban', 'QoS' is 0, and the 'Retenir' (Retain) toggle is off. The 'Payload (modèle autorisé)' (Payload (allowed model)) field contains 'Hello de Chaban'. A 'Publier' (Publish) button is visible.

In the 'Écouter un sujet' section, the 'Écouter' (Listen) toggle is on, and 'Mettre en forme le contenu JSON' (Format JSON content) is checked. The 'Sujet' (Topic) is 'test/chaban' and 'QoS' is 0. An 'Arrêter d'écouter' (Stop listening) button is present. Below, it shows a received message: 'Message 0 reçu sur test/chaban à 10:56 : Hello de Chaban'. At the bottom, it indicates 'QoS: 0 - Retain: false'.

Figure 4 : Test MQTT depuis Home Assistant.

III. Dispositif ESP8266 avec ArduinoHA

A. LED contrôlable via Home Assistant

L'objectif de cette partie est d'utiliser le microcontrôleur ESP8266 comme un objet connecté qu'on pilote depuis Home Assistant. Nous utilisons pour cela la bibliothèque ArduinoHA, qui gère automatiquement la découverte des appareils avec le protocole MQTT.

Le code (Annexe 1) réalisé permet de simuler une lampe intelligente. Il initialise une connexion WiFi puis se connecte au broker MQTT de Home Assistant. Nous déclarons un objet HALight nommé "LED chaban". La fonction onStateCommand est déclenchée lorsqu'on bascule l'interrupteur sur l'interface Home Assistant. Elle commande la broche LED_PIN et renvoie l'état au serveur.

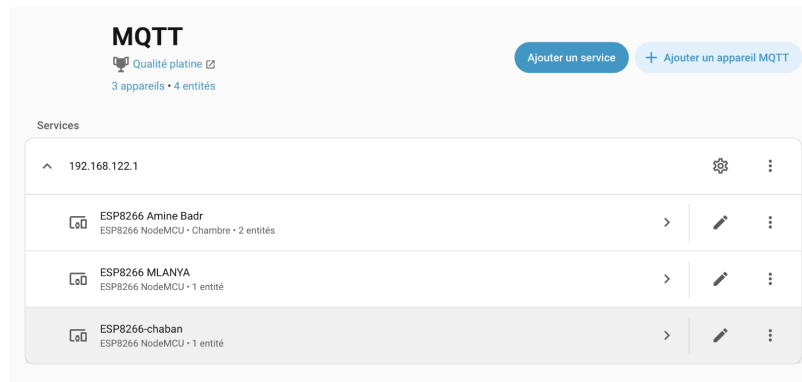


Figure 5 : ESP8266-chaban bien présent dans les paramètres.

Après le téléversement du code, l'ESP8266 communique son existence au broker. Grâce au protocole de découverte automatique d'ArduinoHA, l'appareil apparaît dans l'interface.

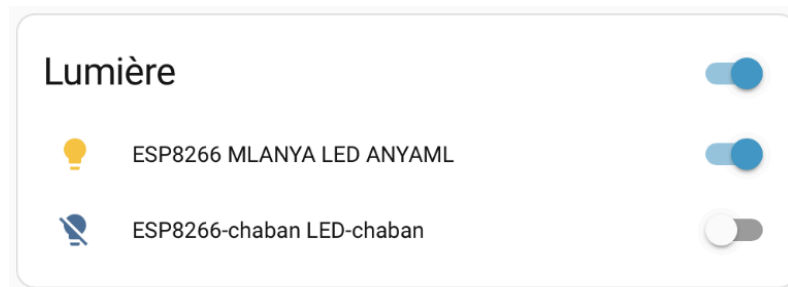


Figure 5 : Apparition de LED-chaban sur l'interface.

Cette manipulation montre qu'au lieu de configurer manuellement chaque topic MQTT, la bibliothèque ArduinoHA permet de présenter l'objet à Home Assistant, qui génère automatiquement l'interface de contrôle (dans notre cas, un bouton ON/OFF).

B. Ajout d'un capteur de luminosité

L'objectif maintenant est d'ajouter un capteur de luminosité à notre dispositif IoT. Nous souhaitons transmettre la luminosité ambiante à Home Assistant et suivre son évolution en temps réel.

Le code (Annexe 1) permet de transformer une tension analogique en une information compréhensible par l'utilisateur. Nous utilisons la classe `HASensorNumber` pour créer une entité nommée "luminosity". La fonction `analogRead(3)` récupère une valeur entre 0 et 1023. La fonction `map` convertit cette valeur brute en un pourcentage (0 à 100 %). Pour ne pas surcharger le réseau et le broker MQTT, l'envoi de la donnée n'est pas continu. La publication est déclenchée toutes les 10 secondes.

Une fois le code téléversé, nous avons pu vérifier l'intégration réussie dans Home Assistant. Comme le montre la Figure 7, une nouvelle carte est apparue pour l'appareil ESP8266-chaban. L'entité "Luminosité-chaban" affiche une valeur en temps réel (72,0 % lorsque la luminosité est élevée, 1 % lorsqu'on simule une baisse de luminosité).

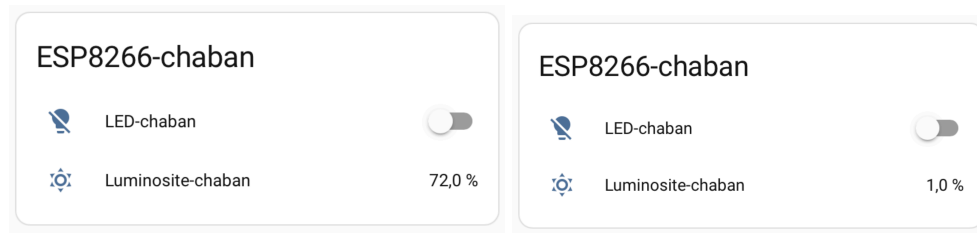


Figure 7 : Affichage de la luminosité.

On peut aussi observer un graphique montrant les fluctuations de la luminosité. Les chutes brusques de la courbe correspondent aux moments où nous avons volontairement masqué le capteur.

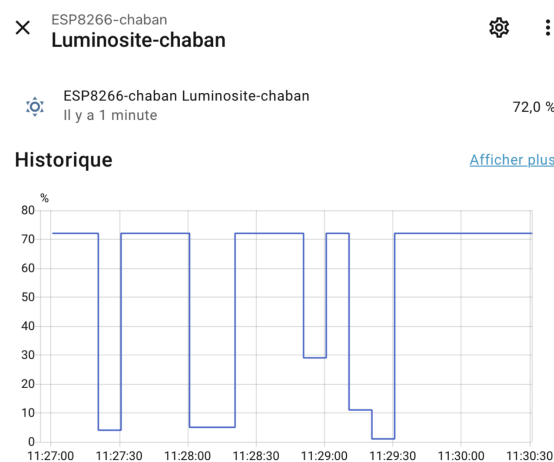


Figure 8 : Variation de la luminosité.

IV. Automatisations

A. Automatisation simple

L'objectif est d'allumer automatiquement la LED lorsque la luminosité devient trop faible. Nous avons configuré l'automatisation suivante.

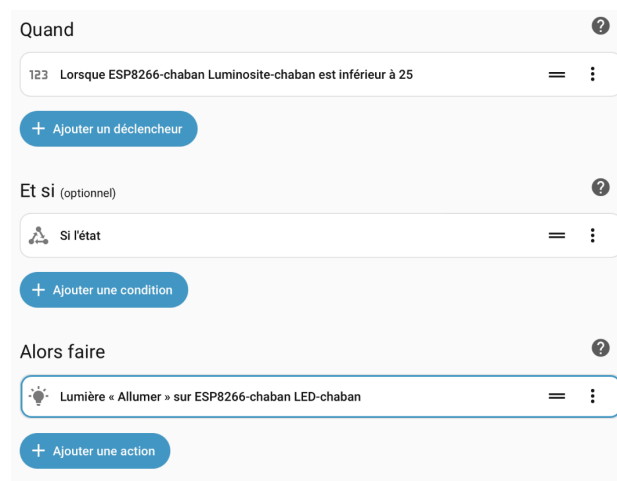


Figure 9 : Automatisation qui permet d'allumer la LED en cas de faible luminosité.

Lors des tests, on constate un problème logique : l'automatisation allume la LED quand il fait sombre, mais ne l'éteint jamais lorsque la luminosité remonte au-dessus de 25. Pour corriger cela, il faudrait soit ajouter un bloc "Si-Alors-Sinon", soit créer une seconde automatisation pour l'extinction.

Quand ?

123 Lorsque ESP8266-chaban Luminosite-chaban est supérieure à 25 = ⋮

+ Ajouter un déclencheur

Et si (optionnel) ?

Si l'état = ⋮

+ Ajouter une condition

Alors faire ?

« Lumière « Éteindre » sur ESP8266-chaban LED-chaban » = ⋮

+ Ajouter une action

Figure 10 : Correction de l'automatisation.

B. Automatisation avancée

Cette automatisation permet de créer un clignotement automatique en utilisant une boucle qui se répète cinq fois. À l'intérieur de cette boucle, on alterne entre l'allumage et l'extinction de la LED, avec une pause d'une seconde entre chaque étape pour que l'effet soit bien visible à l'œil nu.

Quand ?

La valeur de ESP8266-chaban Luminosite-chaban change = ⋮

+ Ajouter un déclencheur

Et si (optionnel) ?

Toutes les conditions ajoutées ici doivent être satisfaites pour que l'automatisation s'exécute. Une condition peut être satisfaite ou non à tout moment, par exemple : « Si user est à la maison ». Vous pouvez utiliser des blocs de construction pour créer des conditions plus complexes.

+ Ajouter une condition

Alors faire ?

⌵ Répéter une action 5 fois = ⋮

Actions:

⚡ Lumière « Allumer » sur ESP8266-chaban LED-chaban = ⋮

⌚ Attendre pendant 1 seconde = ⋮

⚡ Lumière « Éteindre » sur ESP8266-chaban LED-chaban = ⋮

⌚ Attendre pendant 1 seconde = ⋮

+ Ajouter une action

+ Ajouter une action

Figure 11 : Automatisation avancée.

V. Réponse aux questions**Question Architecture : Expliquez le rôle de MQTT dans cette architecture. Pourquoi ne pas utiliser HTTP directement ?**

Dans cette architecture, MQTT sert de protocole de messagerie "léger" entre les objets (ESP) et le serveur (Home Assistant). Le "Broker" (Mosquitto) centralise tous les messages. Pourquoi pas HTTP ? HTTP est un protocole "lourd" (entêtes importants) qui demande plus de ressources et d'énergie. Avec HTTP, pour recevoir un ordre, l'ESP devrait agir comme un serveur web et rester en écoute tout le temps. Avec MQTT, l'ESP reste un client qui reçoit les ordres du broker dès qu'ils sont publiés sur son topic.

Question Sécurité : Quelles mesures de sécurité pourriez-vous ajouter pour protéger la communication entre l'ESP8266 et Home Assistant ?

On pourrait utiliser MQTTS avec TLS (port 8883) pour que les données circulant entre l'ESP et le serveur soient cryptées et illisibles par un tiers. On pourrait aussi ne pas laisser le broker MQTT en accès anonyme et utiliser des identifiants uniques pour chaque appareil. On peut aussi créer des VLAN pour placer tous les objets IoT sur un réseau Wi-Fi séparé du réseau principal pour éviter qu'une faille sur un ESP ne puisse être compromettante pour le reste du réseau.

Question Scalabilité : Comment géreriez-vous 50 ESP8266 dans un bâtiment ? Quelles bonnes pratiques de nommage et d'organisation adopteriez-vous ?

On utiliserait une structure logique, par exemple :

batiment_GEI/etage1/salle102/sensor/luminosite.

Dans Home Assistant, on peut utiliser les fonctions "Pièces" (Areas) et Labels pour regrouper les entités par zone géographique ou par fonction. On peut aussi utiliser des outils comme ESPHome ou des scripts de déploiement pour éviter de configurer chaque code à la main (qui gère automatiquement les mises à jour).

Question Optimisation : Comment réduire la consommation énergétique de l'ESP8266 pour un fonctionnement sur batterie ?

Pour faire fonctionner un ESP8266 sur batterie, il faut limiter son temps d'activité. On peut utiliser le mode Deep Sleep : le processeur s'éteint et ne se réveille (avec un timer par exemple) que pour envoyer sa mesure, avant de se rendormir.

Au lieu d'envoyer la luminosité toutes les 10 secondes, on peut l'envoyer toutes les 10 minutes (dans la vraie vie, elle ne change pas toutes les 10 secondes. C'était le cas dans ce TP uniquement pour le test). On peut aussi l'envoyer seulement si la valeur change de plus de 20 % (par exemple).

On peut aussi configurer une adresse IP statique dans le code pour que l'ESP ne perde pas de temps (et d'énergie) à attendre une réponse du serveur DHCP lors de chaque réveil.

Question Extension : Proposez un cas d'usage réel dans votre domaine d'ingénierie où cette solution serait pertinente.

On peut déployer un réseau de microcontrôleurs (ESP8266) équipés de capteurs de présence (infrarouge), de capteurs d'ouverture sur le réfrigérateur ou le pilulier, et de capteurs de luminosité. Les données sont transmises via MQTT à un serveur Home Assistant local. On analyse les habitudes de vie (La personne s'est-elle levée, a-t-elle ouvert son pilulier ?). Des automatisations sont configurées pour détecter des anomalies. Par exemple, si aucune luminosité n'est détectée et qu'aucun mouvement n'est enregistré dans la cuisine avant 10h du matin, le système considère cela comme une situation à risque (chute ou malaise).

VI. Conclusion

Pour conclure, ces manipulations ont démontré les fonctionnalités de Home Assistant et l'efficacité du protocole MQTT pour la gestion d'objets connectés. Nous avons réussi à établir une communication entre le serveur et l'ESP8266, qui a permis le contrôle manuel et l'automatisation de tâches basées sur des capteurs. Ce TP illustre comment un middleware peut transformer des composants électroniques isolés en un système cohérent, réactif et facilement évolutif.

VII. Annexe : Code Arduino

```
#include <ESP8266WiFi.h>
#include <ArduinoHA.h>

const char* ssid      = "asni";
const char* password = "asniasni";

const char* mqtt_server = "192.168.1.1"; // IP de Home Assistant
const int   mqtt_port   = 1883;
const char* mqtt_user   = "insa";
const char* mqtt_password = "insa";

const int LED_PIN = LED_BUILTIN; // LED intégrée du NodeMCU (logique inversée)
const int LUM_PIN = A0;

byte mac[] = {0x07, 0x03, 0x7B, 0x0A, 0x0B, 0xD0};

WiFiClient espClient;
HADevice device(mac, sizeof(mac));
HAMqtt mqtt(espClient, device);

unsigned long lastUpdate = 0;
const unsigned long updateInterval = 10000;
// Déclaration de la LED comme "Light" avec gestion de la luminosité
HALight led("channe-le-lampadaire", HALight::BrightnessFeature);
HASensorNumber luminosity("lou-capteur", HASensorNumber::PrecisionP1);

void onStateCommand(bool state, HALight* sender) {
    if (state) {
        digitalWrite(LED_PIN, HIGH); // LED ON (logique inversée)
        Serial.println("LED ON");
    } else {
        digitalWrite(LED_PIN, LOW); // LED OFF
        Serial.println("LED OFF");
    }

    sender->setState(state); // Mise à jour de l'état dans HA
}
```

```

void onBrightnessCommand(uint8_t brightness, HALight* sender) {
    // Conversion de la luminosité (logique inversée)
    int pwmValue = map(brightness, 0, 255, 255, 0);
    analogWrite(LED_PIN, pwmValue);

    sender->setBrightness(brightness);

    Serial.print("Luminosité : ");
    Serial.println(brightness);
}

void setup() {
    Serial.begin(115200);

    pinMode(LED_PIN, OUTPUT);
    pinMode(LUM_PIN, INPUT);
    digitalWrite(LED_PIN, LOW); // LED éteinte au démarrage

    /* Connexion WiFi */
    Serial.print("Connexion WiFi");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi connecté !");
    Serial.print("Adresse IP : ");
    Serial.println(WiFi.localIP());

    /* Configuration du device Home Assistant */
    device.setName("ESP8266-chaban");
    device.setSoftwareVersion("1.1.2");
    device.setManufacturer("Mon École");
    device.setModel("ESP8266 NodeMCU");

    /* Configuration de la LED */
    led.setName("LED-chaban");
    led.onStateCommand(onStateCommand);
    led.onBrightnessCommand(onBrightnessCommand);

    luminosity.setName("Luminosite-chaban");
    luminosity.setUnitOfMeasurement("%");
}

```

```
    luminosity.setIcon("mdi:weather-sunny");

    /* Connexion MQTT */
    mqtt.begin(mqtt_server, mqtt_port, mqtt_user, mqtt_password);
}

void loop() {
    mqtt.loop();
    unsigned long now = millis();
    if (now - lastUpdate > updateInterval) {
        lastUpdate = now;
        int raw = analogRead(LUM_PIN); // 0-1023
        float percent = map(raw, 0, 1023, 0, 100);
        luminosity.setValue(percent);
        Serial.print("Luminosité : ");
        Serial.print(percent);
        Serial.println(" %");
    }
    // Reconnexion WiFi si nécessaire
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("WiFi déconnecté, reconnexion...");
        WiFi.begin(ssid, password);

        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
        }
    }
}
```